Project Journal

Real-Time Bayesian Experimentation Engine

7–20 October 2026

Abstract

This journal records, day by day, how a Bayesian experimentation engine for choosing checkout offers on a quick-commerce app was designed, built and evaluated. It covers the experiment design, the traffic simulator, the statistics behind each method, the bugs found along the way, and the measured results of five decision methods across three scenarios. All data is simulated, so every method is scored against a known truth.

Contents

7 October: The first plan **1**

15 October: Pivot to experimentation **1**

18 October: Experiment design and the simulator **2**

19 October: Simulator, first tests and the fixed A/B baseline **3**

20 October: Bayesian methods and the final comparison **4**

7 October: The first plan

planning

Discussion

The project began as a Bayesian demand-forecasting and inventory plan built on a Poisson–Gamma model and the Walmart M5 dataset. On review, the plan was set aside for three reasons:

- M5 is stored in wide format and is highly intermittent, with many zero-sales days.
- Sales are censored demand: a shelf that is empty records zero sales even when customers want the product.
- The inventory side would have to be simulated anyway, and the plan assumed more modelling experience than the starting point, which was Beta–Binomial updating.

15 October: Pivot to experimentation

planning, research

Discussion

An earlier project (DataFlow) read as backend engineering rather than data science: it posed no question, fitted no model and drew its insights from data it had generated itself. The new project

is a **Bayesian experimentation engine** that chooses which checkout offer to show on a quick-commerce app. It builds directly on Beta updating, and experimentation is a core skill for data science and product analytics roles. Target date: 25 October.

Research

- Fixed-horizon A/B tests remain the industry default. Checking results repeatedly ("peeking") inflates false positives (Evan Miller).
- Industry responses include sequential tests (Spotify), always-valid inference (Optimizely) and variance reduction with pre-experiment data (CUPED at Microsoft Bing, about 50% less variance).
- Bayesian A/B testing with an expected-loss stopping rule is used in commercial tools (VWO). Thompson sampling (Thompson, 1933; Chapelle and Li at Yahoo, 2011) powers bandit systems at Google, Netflix, Stitch Fix and JD.com.
- Bandits are good at earning while learning and weaker at measuring effect sizes. A fair study must compare them with a proper A/B test, not a weakened one.

18 October: Experiment design and the simulator

design

The experiment

Which checkout offer should a campaign show? The unit is one cart-page visit; the outcome is whether the visitor orders and, if so, the profit earned. The toy values below are assumptions, not real data.

Offer	Description	True order rate	Profit per order	Profit per visitor
0	No offer (control)	0.08	Rs. 90	Rs. 7.20
1	Free delivery	0.10	Rs. 70	Rs. 7.00
2	Rs. 50 off	0.12	Rs. 40	Rs. 4.80
3	Rs. 20 off above 299	0.11	Rs. 70	**Rs. 7.70**

Offer 2 converts best but earns least, and offer 3 is the true profit winner. A method that judges by conversion alone picks the wrong offer.

Design decisions

- The truth is one order rate per offer, hidden from every decision method and used only for scoring.
- Three ideas must be kept apart: the true rate p (fixed and hidden), the outcome of a visit (a Bernoulli draw), and the belief about p (a Beta posterior).
- Returning users must always see the same offer. Assignment uses a salted hash, hspace*1emsha256("user_id:experiment_id") mod 4; plain `user_id % 4` would give two experiments identical splits, so their effects could not be separated.
- Arrivals follow a non-homogeneous Poisson process: each minute has its own rate λ_t. The daily total is Poisson with mean $\sum_t \lambda_t$, but the timing comes from how λ_t varies.
- "Returning-visit share" (the share of visits from someone already seen in the window) differs from the customer-level "repeat purchase rate". A share of 0.15 is used as an assumption.

Calculations

$$\text{relative spread of a Poisson count} = \frac{\sqrt{\lambda}}{\lambda} = \frac{1}{\sqrt{\lambda}} \qquad (\lambda = 40:\ 16\%;\ \lambda = 2400:\ 2\%)$$

$$\text{expected distinct users (pool } M\text{, } N \text{ visits)} = M\left(1 - e^{-N/M}\right)$$

$$\text{share of visits from someone already seen} = 1 - \frac{1}{\text{average visits per active user}}$$

Daily traffic

Daypart shares follow ranges from an industry article (Zane, 2 August 2026), which cites no dataset and is therefore treated as a plausible range: 07–11: 16%, 11–17: 24%, 17–23: 46%, 23–02: 10%, and an assumed floor of 4% for 02–07. Volume is about 20,000 visits per day.

19 October: Simulator, first tests and the fixed A/B baseline

code, results

The per-visit simulator

- Bugs found and fixed: a NumPy shape passed as a data type; `rng.shuffle` returning `None`; `np.append` not changing the array in place (a silent bug that made the returning share 0%); a hidden cap of 100,000 user IDs that would run out on day six; and Poisson counts drawn once and copied to every day.
- Verified over 3 days (seed 42, 60,374 visits): visits per day 20,320, 19,884 and 20,170; window shares 4.0, 16.1, 24.0, 46.0 and 9.9%; returning share 15.1%; no user saw more than one offer.

The two-proportion z-test

$$\bar{p} = \frac{x_1 + x_2}{n_1 + n_2}, \qquad SE = \sqrt{\bar{p}(1-\bar{p})\left(\tfrac{1}{n_1} + \tfrac{1}{n_2}\right)}, \qquad z = \frac{\hat{p}_2 - \hat{p}_1}{SE}, \qquad p\text{-value} = 2\,\Phi^c(|z|)$$

- The rate is pooled because, under the null hypothesis, both groups share one rate.
- z grows with $\sqrt{n}$: ten times the data gives about 3.16 times the z.
- The survival function `norm.sf` keeps precision in the far tail, where `1 - norm.cdf` rounds to zero.
- A p-value is the probability of data this extreme if the null is true, not the probability that the null is true.

Tests on one 3-day run

Comparison	Result	Note
Conversion, offer 2 vs control	$z = 9.35$	matches statsmodels; $\chi^2 = z^2 = 87.43$
Conversion, offer 3 vs control	$z = 7.00$	95% interval (1.71, 3.04) points
Profit per visitor, offer 3 vs control	$t = -0.10$, $p = 0.92$	a true gap of Rs. 0.50 is invisible

Welch's t-test from totals

Each visitor earns either 0 or the offer's fixed profit m, so the test needs only totals:

$$\text{mean} = m\,p, \qquad \text{variance} = m^2\,p(1-p), \qquad t = \frac{\bar{x}_k - \bar{x}_0}{\sqrt{s_k^2/n_k + s_0^2/n_0}}.$$

Detecting a Rs. 0.50 gap with 80% power at the 5% level needs $n \approx (1.96 + 0.84)^2(\sigma_0^2 + \sigma_3^2)/0.5^2 \approx$ 34,000 visitors per offer, about seven days of all traffic.

The fast simulator and policy classes

Hourly batches replace the per-visit loop: visits $\sim$ Poisson(hourly rate), the split across offers $\sim$ Multinomial, and orders $\sim$ Binomial. Each decision method is a class with four methods (`allocate`, `update`, `should_stop`, `choice_test`), so the simulation loop never changes. Monte Carlo runs repeat each campaign 500 times with independent seeds from `SeedSequence(42).spawn`.

Fixed A/B test: results

Fixed A/B test	3 days	14 days
Normal: picks offer 3	32.6%	95.4%
A/A: declares a non-control winner	1.4%	2.2%
Tiny gap: picks offer 3	3.6%	16.2%

All results agree with theory: about 30% and 95% power in the normal scenario, about 4% and 15% in the tiny-gap scenario, and about 2% false alarms at either length. The A/A rate follows from the decision rule: each comparison needs $t > 2.39$ (two-sided 0.05/3, "better" direction only), which happens with probability 0.83%; three comparisons that share a control group give about 2%.

Peeking

Peeking means running the test every evening and stopping at the first significant result. Each extra look is another chance for noise to cross the threshold. Measured over 14 days, false alarms rise from 2.2% to 9.4%, while regret falls by 62% because the large true gap is usually found early.

Notes

- Monte Carlo simulation (repeating a random experiment and counting outcomes) is not MCMC (Markov chain Monte Carlo, a method for sampling posterior distributions).
- Regret is the profit lost compared with always showing the best offer. The best offer maximises rate $\times$ profit, not profit alone. In A/A every offer is best, so regret is always zero and A/A is judged by false alarms instead.

20 October: Bayesian methods and the final comparison code, results

Bayesian expected-loss stopping

Instead of asking whether a gap is significant, this method asks: *if we pick this offer now and we are wrong, how much profit per visitor do we expect to lose?* Each offer's order rate has the posterior Beta(1 + orders, 1 + visits − orders). The method draws 5,000 possible worlds, multiplies each rate by its profit per order, measures each offer's loss against the best offer in every world, and averages. Each evening it stops if the smallest expected loss is below Rs. 0.01 per visitor. The sampling error of this estimate is about Rs. 0.0003, well below the threshold.

Thompson sampling

Thompson sampling changes the traffic split while the test runs: every hour, each offer receives a share of traffic equal to its probability of being the best offer, estimated as the share of 200 posterior worlds it wins. Poor offers lose their traffic within hours, while uncertain offers keep some, so exploration follows uncertainty. In one campaign, offer 2 received 409 visitors in three days instead of about 15,000. The final method combines Thompson's traffic split with the expected-loss stopping rule.

Bugs fixed

Beta parameters fixed at (1,1) instead of updated from data; comparing order rates instead of profit; averaging the whole loss table instead of each column; an attribute that shadowed a method of the same name; comparing arrays inside `if`; storing a reference to a mutable array instead of its total; and global variables that leaked from one campaign to the next.

Final results (500 campaigns per cell)

Normal scenario	Fixed	Peeking	Bayesian	Thompson	Thompson + loss
Picks offer 3, 3 days	32.6%	39.0%	97.2%	98.0%	**98.6%**
Picks offer 3, 14 days	95.4%	96.6%	99.6%	**100%**	99.2%
Regret, 3 days (Rs.)	61,524	54,631	47,473	8,738	**8,355**
Regret, 14 days (Rs.)	287,006	110,267	62,144	13,430	**10,197**
Profit per visitor, 14 days	6.68	7.31	7.48	7.65	**7.66**
Conversion, 14 days	10.3%	10.7%	10.8%	10.8%	10.9%
A/A: non-control pick, 14 days	2.2%	9.4%	73.4%	74.8%	75.2%
Tiny gap: picks offer 3, 14 days	16.2%	26.2%	77.0%	85.8%	**86.0%**
Tiny gap: regret, 14 days (Rs.)	37,801	33,747	31,672	23,138	**22,670**

Interpretation

- The combined engine loses about 28 times less profit than the fixed A/B test over 14 days and earns within 0.5% of the best possible profit per visitor (Rs. 7.70).
- In a 3-day campaign, every Bayesian method finds the best offer at least 97% of the time, against 33% for the fixed test.
- No method chose offer 2: decisions on profit avoid the conversion trap.
- In A/A, the Bayesian methods pick among identical offers at random, at no cost, and stop early in only 21–24% of campaigns (median day 8). They are decision tools, not tools for certifying an effect.
- With a 2.5% gap, the fixed test finds the better offer in 16% of 14-day campaigns; Thompson-based methods find it in 86%.

Speed

Per 14-day campaign: fixed 5 ms, peeking 4 ms, Bayesian 7 ms, and Thompson 183 ms with 1,000 worlds per hour, reduced to 27 ms by using 200 worlds.

Future scope

- Run campaigns in parallel across CPU cores (about ten times faster, identical results).
- Simulate all campaigns at once as arrays (50–100 times faster; requires rewriting the loop and policies).

- Serve the engine live: each offer's posterior held in Redis, with FastAPI endpoints to assign an offer and record an outcome.